



DALHOUSIE UNIVERSITY

CSCI 5409/4177: Advanced Topics in Web Development

Project Proposal

Group: 07

Team Members: Sanif Ali Momin - B00979961
Isabella Ashton - B00939781
Alberto Magana -B01075922
Ryan Blenkhorn-Hyde - B00934745
Mostafaa Abdelaziz - B00875982

Version: 02
Date: May 29, 2026

Contents

1. Title: Give your project topic a concise and descriptive title.	2
2. Team members: List the names (and Banner ID) and key responsibilities of each group member.	2
3. Problem Statement and Motivation: This section should address the following key questions:	2
a. What problem are you solving with your web app? (discuss the real-world problem that your app will address)	2
b. Why does it matter? (discuss user pain-points, business value, etc. to show the importance of your project)	3
c. Who is affected? (discuss your target users)	3
4. Objectives: Outline the objectives of your project.....	3
5. Scope and Deliverables: Discuss the core features and the AI-enabled feature that your proposed app will provide. Describe each feature in detail.	3
6. Technical Approach: Which front-end framework (e.g., React), backend framework (e.g.,Node.js+Express) and database system(e.g.,MongoDB) do you intend to use in developing the app? Do you plan to use external services, libraries, or APIs for any/some of your features - which ones and why do you need those external services?	4
7. System Architecture: Include a simple diagram showing how the main parts of your web application will work together. Your diagram should show the frontend, backend and API, database, any external services, user roles, core features, and the AI enabled feature.	5
8. Timelines and tasks: Create an 8 week plan with clear milestones and timelines..	5

1. Title: Give your project topic a concise and descriptive title.

DalMarketplace - Online Marketplace for Dalhousie Students

2. Team members: List the names (and Banner ID) and key responsibilities of each group member.

Isabella Ashton, B00939781 - Front-end developer responsible for designing High-Fidelity prototypes. Will provide support in developing the user interface on React. Also responsible for quality assurance testing, bug finding, and fixing throughout development.

Alberto Magana, B01075922 - Front-end developer responsible for developing the user interface of the application using React. This includes building reusable components, connecting the front end with backend services, improving page responsiveness, and testing the design, navigation, and overall usability from the user's perspective.

Ryan Blenkhorn-Hyde, B00934745 - Backend developer responsible for handling user authentication and security. This includes verifying Dalhousie student status (current or former), handling storage and access of user account information, handling logging in to their accounts on the web app, and authorizing signed-in users to access the app's features while restricting them from others.

Mostafaa Abdelaziz B00875982 - AI and messaging integration lead- will integrate the Gemini API for AI-generated listing summaries and the Stream Chat API for in-app messaging, while also testing prompt quality, summary accuracy, and chat usability. Integrating Cloudinary API for image upload and storage.

Sanif Ali Momin B00979961 - Backend developer responsible for designing the MongoDB database schema and building the core listing, search, and review APIs using Node.js and Express. This includes implementing keyword search with category, region, and price filters, and developing the seller review and star rating system. Also responsible for deploying and configuring the backend on Render and the database on MongoDB Atlas.

3. Problem Statement and Motivation: This section should address the following key questions:

a. What problem are you solving with your web app? (discuss the real-world problem that your app will address)

- Creating a safe space for current and alumni Dalhousie Students to sell or buy things off other students and reducing the "Stranger Danger" feeling and knowing that the seller or buyer is registered in the Dalhousie System which provides a sense of verifiability.

- At the end of an academic year, Students move out of residence and throw away a lot of their belongings due to not having enough space or knowing where to sell. Eliminating excess waste of items can be reduced and actually benefit other students who are looking to move in and get these stuff. The DalMarketplace allows students to list their items, accessories, and their rooms for rent/sale in order to make their lives easier.
- Many students are living on a tight budget when they're studying or planning to go to Dalhousie and cannot afford brand new furniture or technologies, DalMarketplace allows for cheaper (if not better) alternatives that can be found in the student community.

b. Why does it matter? (discuss user pain-points, business value, etc. to show the importance of your project)

Students already try to sell unneeded items, from clothes to furniture to subletting apartments, and attempt to do so on various unrelated apps such as snapchat and instagram, which are not built for this purpose. This app would help to eliminate issues that occur due to these app's lack of features. Students have already displayed that they need similar products. Additionally, the staff found in residence communities would end up having to clean and dispose of those extra unnecessary stuff because students couldn't find an alternative for it and causes extra workload on custodial workers and space that isn't available in these buildings.

c. Who is affected? (discuss your target users)

The primary sellers on this platform are students (or alumni) who create or resell products, handmade crafts, custom artwork, second-hand furniture, and vintage clothing, who need a low-cost, easy-to-use platform to reach customers. The primary buyers are students looking for affordable alternatives to retail prices. They want to support peers, discover locally made products, and transact with people they can identify with and trust.

4. Objectives: Outline the objectives of your project.

Create profiles for students, you must use the B00 number in profile. All users can both post and purchase items. Sellers are able to post pictures of their items, a description, location, and other necessary details. Buyers are able to browse all items based on location and search prompt to discover items. Sellers and buyers are able to communicate with each other about the purchase of their item. AI will be utilized to help simplify and provide an overview of the item description and features.

5. Scope and Deliverables: Discuss the core features and the AI-enabled feature that your proposed app will provide. Describe each feature in detail.

- **Core Feature 1:** Verified Dalhousie user access: Only current or former Dalhousie students will be allowed to create accounts and fully use the marketplace. During registration, users will provide identifying information tied to a Dalhousie-based account or approved alumni verification process, and the system will store a banner ID in the user profile. This feature matters because the project's main value is trust: buyers and sellers should know they are transacting within a verifiable university community rather than with unknown strangers.

- **Core Feature 2:** Search and discovery: Buyers will be able to browse all active listings and search by keywords such as “desk,” “monitor,” or “sublet.” Search results will be filtered using listing titles, descriptions, and categories so students can quickly find relevant results instead of scrolling through unrelated posts.
- **Core Feature 3:** In-app chat between buyer and seller: Each listing will include a way for interested users to message the seller directly through the app. The chat system will let buyers ask questions about condition, price, pickup time, or availability without exposing personal contact information publicly in the listing itself. This improves both privacy and usability compared with using general-purpose social apps.
- **Core Feature 4:** Seller reviews and ratings: After a transaction, buyers will be able to leave a star rating and short written review for the seller. These reviews will help future buyers judge whether a seller is reliable, responsive, and honest about item condition. Over time, this creates accountability and strengthens trust within the Dalhousie marketplace community.
- **AI enabled Feature 1:** AI-generated listing summary: When a seller submits a listing, the system will send the listing title and full description to the Gemini API to generate a short, readable summary of the item. The summary will restate the item type, highlight the most important details, and mention condition information when it is available in the seller’s text. This feature is intended to improve readability for buyers, especially when listings are poorly written, too brief, or overly long, while keeping the seller’s original description visible as the main source of truth
- **Core Feature 5:** Listing status tracking: Sellers will be able to change a listing’s status to available, reserved, or sold. This reduces wasted messages about items that are no longer obtainable and helps keep the marketplace current and useful during busy move-in and move-out periods.

6. Technical Approach: Which front-end framework (e.g., React), backend framework (e.g., Node.js+Express) and database system(e.g., MongoDB) do you intend to use in developing the app? Do you plan to use external services, libraries, or APIs for any/some of your features - which ones and why do you need those external services?

Frontend: React will be used to build the web interface, including pages for login, profile creation, listing browsing, listing creation, chat, reviews, and filters.

Backend: Node.js with Express will be used to implement the server-side API, including authentication-related checks, listing CRUD operations, review handling, search endpoints, and AI-summary requests

Database: Firebase Firestore will store users, listings, reviews, chat references, and status metadata

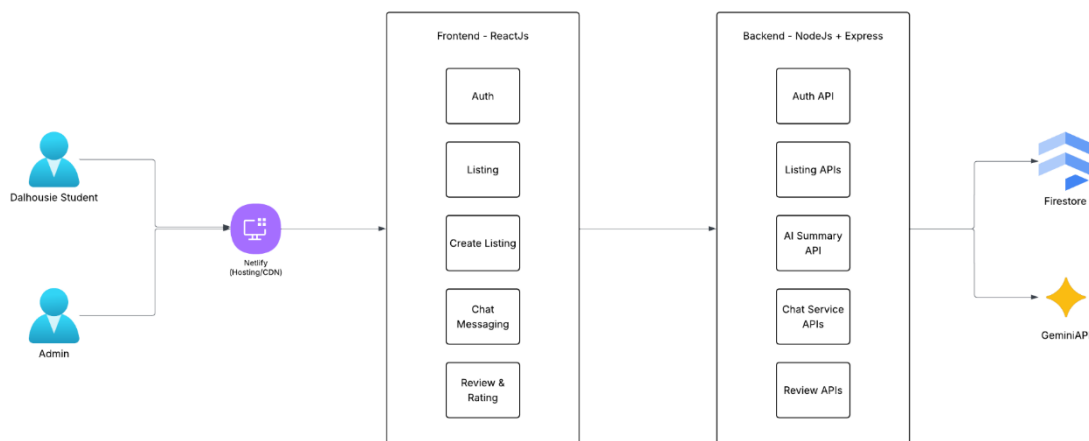
Deployment: Netlify will host the frontend application for public access

Google Gemini API (Free version limited): This API will be used for the AI-enabled feature that generates a concise listing summary from the seller’s title and description. It is needed to provide automated summaries that improve listing readability and help buyers understand key item details faster.

GetStreamChat API: This API will be used to support real-time in-app messaging between buyers and sellers.

Cloudinary API: This API will be used for Image upload, storage, and optimization for listing photos

7. System Architecture: Include a simple diagram showing how the main parts of your web application will work together. Your diagram should show the frontend, backend and API, database, any external services, user roles, core features, and the AI enabled feature.



8. Timelines and tasks: Create an 8 week plan with clear milestones and timelines.

Week	Milestone	Tasks
Week 1	Project Setup & Design	- Create GitHub repository and set up branch protection rules - Initialize React (Vite) frontend and Node.js/Express backend as separate projects - Configure ESLint, Prettier, and project folder structure - Design MongoDB schema for users, listings, reviews collections - Create low-fidelity wireframes in Figma for all key pages (home, browse, listing detail, post listing, profile, messages)

		• Create high-fidelity Figma mockups with final colour scheme, typography, and component design • Register for Gemini API, Stream Chat API, and Cloudinary accounts; obtain API keys
Week 2	Authentication & User Profiles	• Implement user registration endpoint with Dalhousie email validation (@dal.ca / @dalu.ca) • Implement email verification flow (send token via email, verify on click) • Implement JWT-based login and logout • Build protected route middleware on the backend • Build registration, login, and email verification pages on the frontend • Build user profile page (display name, B00 number, profile picture, listings history) • Implement Cloudinary profile picture upload
Week 3	Listing Creation & Browse	• Build listing creation form (title, description, price, condition, category, region, photo upload) • Implement Cloudinary multi-image upload (up to 5 photos per listing) • Build backend CRUD API routes for listings (POST /api/listings, GET /api/listings/:id, PUT, DELETE) • Build listing browse page (paginated grid of active listings) • Build listing detail page (all listing info, seller info, AI summary box, message button) • Build "My Listings" dashboard with listing status management (Available / Reserved / Sold)
Week 4	Search, Filters & Location	• Create MongoDB text index on listing title and description fields • Implement keyword search API endpoint (GET /api/search?q=...) • Implement filter parameters in search API: category, region, price range, condition • Build search bar and filter sidebar/panel on the frontend • Implement region tagging on listing creation (HRM, Nova Scotia, other province) • Connect frontend filter state to backend search queries with debounced API calls • Implement sort options (Most Recent, Price Low–High, Price High–Low)

Week 5	AI Integration & Chat Messaging	- Integrate Gemini API: on listing creation, send title + description + category + condition to Gemini; store the returned summary in the listing document in MongoDB - Display AI summary in a highlighted card on the listing detail page - Add "Regenerate Summary" button that recalls Gemini when a seller edits their listing - Integrate Stream Chat API: generate user tokens on the backend (POST /api/chat/token) - Build messaging UI on the frontend using the Stream Chat React SDK - Implement "Message Seller" button on listing detail page that creates or opens a channel tied to that listing - Build unified messaging inbox page for viewing all active conversations
Week 6	Reviews, Ratings & Integration Testing	- Build review submission form (star rating + comment) on seller profile pages - Implement backend review API (POST /api/reviews, GET /api/reviews/seller/:id) - Add gate: only users who have an active chat conversation about a sold listing can submit a review - Display average star rating on seller profiles and beside seller names on listing cards - Integrate all features end-to-end: run through full user flows (register → post listing → browse → message seller → mark sold → leave review) - Fix integration bugs discovered during testing - Implement review flagging (report abusive reviews)
Week 7	Testing, Bug Fixes & Polish	- Conduct full end-to-end testing of all user flows across different browsers (Chrome, Firefox, Safari) - Test responsive design on mobile viewport sizes - Address all bugs found during testing - Add loading states and error states to all async UI components - Perform input validation review (ensure all forms are protected against empty/malformed input) - Conduct security review: check JWT expiry, ensure API keys are in environment variables, confirm no sensitive data is exposed in frontend

		Polish UI: spacing, typography consistency, empty state illustrations, 404 page
Week 8	Deployment & Presentation	- Deploy MongoDB Atlas cluster (production environment) - Deploy Node.js/Express backend to Render (set production environment variables) - Deploy React frontend to Vercel (set API base URL to production backend) - Perform smoke testing on all production URLs - Seed production database with sample listings for demo purposes - Record a short demo video of the full user flow - Prepare presentation slides covering: problem, solution, architecture, feature walkthrough, and technical challenges